

Graph based Nearest Neighbor Search: Promises and Failures

Peng-Cheng Lin, Wan-Lei Zhao

Abstract—Recently, graph based nearest neighbor search gets more and more popular on large-scale retrieval tasks. The attractiveness of this type of approaches lies in its superior performance over most of the known nearest neighbor search approaches as well as its genericness to various metrics. In this paper, the role of two strategies, namely hierarchical structure and graph diversification that are adopted as the key steps in the graph based approaches, is investigated. We find the hierarchical structure could not achieve “much better logarithmic complexity scaling” as it was claimed in the original paper, particularly on high dimensional cases. Moreover, we find that similar high search speed efficiency as the one with hierarchical structure could be achieved with the support of flat k -NN graph after graph diversification. Finally, we point out the difficulty, that is faced by most of the graph based search approaches, is directly linked to “curse of dimensionality”.

Index Terms—nearest neighbor search, graph based approach, NN-Descent, high dimensional.

I. INTRODUCTION

NEAREST neighbor (NN) search is a task that searches for the closest neighbors from a group of given candidates for a query. It plays fundamental role in various applications, such as multimedia information retrieval, data mining, computer vision and machine learning. The early exploration about this issue could be traced back to 1970s [1]. However, it is still a new issue in the sense that it becomes more and more imminent recently given the big data problem arises particularly from various multimedia applications. This leads to the timely demand in scalable and efficient nearest neighbor search approach on large-scale and high dimensional data. This decades-old issue is therefore shed with new light. However, due to the “curse of dimensionality”, effective solution is still slow to occur.

In the nearest neighbor search, both the query and the candidate samples are assumed to be from the same space *i.e.*, R^d . The closeness between samples is usually predefined by a metric $m(\cdot, \cdot)$. The time complexity of brute-force search is $O(n \cdot d)$, where n is the number of candidates. Although it is linear with respect to either n or d . Its computational complexity is very high when both n and d are large. Research on this issue aims to explore approach whose time complexity is much lower than $O(n \cdot d)$.

This issue has been traditionally addressed by a variety of tree partitioning approaches, such as K-D tree [1], and R-tree [2], etc. Although they are different in details, they

are all designed to partition the space into hierarchical sub-spaces. The nearest neighbor search only traverses over a few branches of the sub-spaces. Unfortunately, unlike B-tree in 1D case, the partition scheme does not exclude the possibility that nearest neighbor resides outside of these candidate sub-spaces. Therefore, extensive probing over large number of branches in the tree becomes inevitable. Recent indexing structures FLANN [3] and Annoy [4] partition the space with hierarchical k -means and multiple K-D trees respectively. Although they are efficient, sub-optimal results are achieved.

Apart from tree partitioning approaches, quantization based approaches [5], [6] and locality sensitive hashing (LSH) [7] have been extensively explored in the last decade. Approaches from both categories save up lot of memory consumptions and are very efficient when the size of encoding bits is short. Nevertheless, the memory efficiency and speed-up achieved by approaches from both categories are very limited when the search accuracy is required to be high. Another major disadvantage for these approaches is that they are mostly only suitable for l_p -norms. Furthermore, the design of generic hash functions is non-trivial.

Recently, graph based approaches such as nearest neighbor descent (NN-Descent) [8] or hill-climbing (HC) [9], demonstrate superior performance over other categories of approaches in many large-scale NN search tasks [10], [11]. All the approaches in this category are built upon an approximate k -NN graph. The search procedure starts from a group of random seeds and traverses iteratively over the graph by the best-first search. Guided by the neighbors of visited vertex's, the search procedure descends closer to the true nearest neighbor in each round until no better candidates could be found. Approaches in [9], [10], [12] in general follow the similar search procedure. The major difference between them lies in the structure of k -NN graphs upon which the NN search is undertaken. For most of the graph based approaches, the extra merit is that they are suitable for various distance measures.

On one hand, graph-based approaches get more and more popular recently. On the other hand, the overall behavior about them is yet unclear. In this paper, a comprehensive experimental study about graph based approaches is presented. The most popular NN search approaches in recent years, namely hierarchical navigable small world (HNSW) graphs [10] and diversified proximity graph (DPG) [11] are taken as the study cases. Firstly, we confirm that the graph based approaches are apparently superior than approaches of other categories on large-scale datasets of various types. Moreover, we show where the graph based NN search works and where it fails. Namely, we study the role of hierarchical structure and graph

Peng-Cheng Lin and Wan-Lei Zhao are with the Department of Computer Science, Xiamen University, Xiamen, China. Wan-Lei Zhao is the corresponding author. E-mail: wlzhao@xmu.edu.cn

Manuscript received Month Day, Year; revised Month Day, Year.

diversification in the graph based approaches. Contradictory to the claim made in [10], we find the hierarchical structure is not helpful for the nearest neighbor search when the data dimension goes high. Similar or even better performance could be achieved by diversified flat graph. Although graph-based approaches are effective in many scenarios, we further point out that all the graph-based approaches face the same difficulty as the data dimension is high. This difficulty is directly linked to the “curse of dimensionality”.

II. RELATED WORK

There is a significant literature on the nearest neighbor search algorithms as the result of more than four decades of continuous research. They in general fall into four categories. Namely, they are tree-based space partition approaches, quantization-based approaches, locality sensitive hashing approaches and graph-based approaches. In this section and the next, the general idea as well as the advantages and pitfalls of approaches in each category are reviewed.

A. Tree-based Space Partition Approaches

The basic idea of this category is to partition the space into embedded hierarchical sub-spaces. This search process is conducted efficiently by using the tree properties to eliminate large portions of the branches. One commonly used method, the K-D tree [1] is very efficient in low dimensional case. However, as the dimension increases, the performance of K-D tree degrades quickly since the query process has to visit nearly every nodes in the tree. Vantage point tree (VP-tree) [13] is proposed to enhance its performance by introducing a better partition strategy. FLANN [3] and Annoy [14] are two representative space partition algorithm proposed in recent years. FLANN partitions the space via hierarchical k -means. While Annoy partitions the space by a variant of hierarchical k -means. Although they are efficient even on high dimensional data, sub-optimal results are achieved. In our comparative study, both FLANN and Annoy are selected as the representative tree-based approaches.

B. Quantization-based Approaches

Quantization-based search approaches have been extensively explored in recent years [5], [6]. In general, the candidate vectors are all compressed via vector (or sub-vector) quantization. This makes it possible to hold the whole large-scale candidate dataset in the memory. The NN search is conducted between the query and the quantized candidate vectors. The distance between query and candidates is approximated by the distance between query and vocabulary words that are used for quantization. Due to the considerable compression loss on the candidate vectors, high search quality is hardly desirable. In our study, product quantization [5] is selected as the representative approach of this category.

C. LSH-based Approaches

Approaches of this category are derived from the same idea of locality sensitive hashing [7]. Multiple hash tables are

predefined to map vector into a series of hash codes. The hash codes of the query sample are compared to the hash codes of all the candidate vectors. Vectors closer to a query are expected to share the same or similar hash codes as the query. These closer candidates are kept to compare with the query vector directly. In this sense, the hashing acts as a filtering scheme for efficient computation.

In general, more number of hash tables leads to preciser hash encoding, while inducing higher computation and memory consumption. In addition, the performance is highly dependent on the design of hash functions. In the early literatures [15], the efforts are devoted to propose various LSH families for different types of metrics. Later, learning-based hashing approaches [16] take the data distribution into account. Most of these approaches require large amount of space for indexing that limits the scalability of the approach. Recently, a projection-based approach named SRS [17] is proposed. It requires only a single tiny index. SRS shows high scalability and is the state-of-the-art in this category of techniques. Unfortunately, it is only suitable for l_2 -space. In our study, SRS is selected as the representative approach of this category.

D. Graph-based Approaches

Recently, graph-based approaches [8], [10], [12], [11] have drawn considerable attention. Among all these approaches, the key step lies in the construction of the graph structure. A candidate vector is represented as vertex in the graph. Two vertices are connected by an edge when they are sufficiently close from one to another. Usually several tens of neighbors are kept for each vertex. Once the graph is constructed, most of the approaches follow similar search procedure [9]. It starts from one or several random seeds (vertices) and explores the neighborhoods of visited vertices. The search descends to closer samples if any closer samples are discovered. The process repeats until no closer samples are found. As demonstrated in different contexts, graph-based approaches [8], [10], [11], [12] show considerably superior performance over approaches of other categories.

Since the focus of this paper is to study the behavior of graph-based approaches. A more detailed review on the representative approaches in this category is presented in the next section.

III. REVIEW ON GRAPH-BASED NEAREST NEIGHBOR SEARCH

As pointed out in the previous section, most of the graph-based approaches follow similar search procedure. The major difference between them lies in the construction of graph, upon which the nearest neighbor search is undertaken. The most representative graph construction approaches are KGraph [8] and HNSW [10], both of which are generic to various distance measures.

k -nearest neighbor Graph (KGraph) [8]. The idea of KGraph is inspired by the observation that “a neighbor of a neighbor is also likely to be a neighbor”. It starts by generating k random neighbors for each sample. The construction process

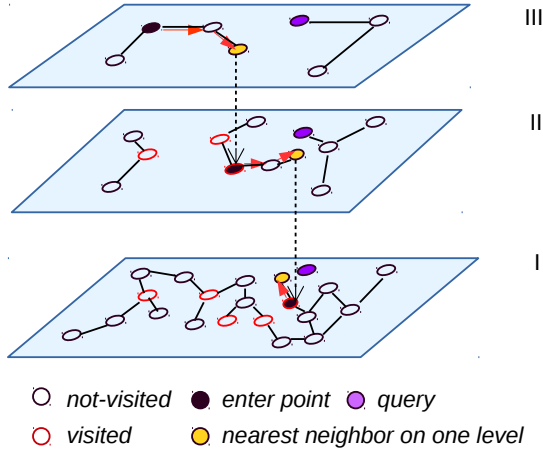


Fig. 1: The illustration of hierarchical structure of HNSW (three layers case). Each layer of HNSW is an NSW graph. The long-range links are maintained on top layers. The search starts from the top layer. The found nearest neighbor from non-bottom layers is treated as the enter point of the NN search on the lower layer.

iteratively refines this raw k -nearest neighbor graph by performing pair-wise comparison within each k -NN and reverse k -NN list. The process terminates when no improvement could be made. In the search step, the hill-climbing search strategy [9] is adopted.

During the hill-climbing search, samples in the visited k -NN list will guide the search to closer neighborhood of the query. As observed in [11] and [18], if samples in the list are close to each other, it will guide the search to reach to the same location. As a result, it is redundant to compare the query to all the neighbors of a visited vertex. Diversified proximity graph (DPG) [11] is proposed recently to enhance the sparsity of the neighborhood. Basically, it removes the samples from the neighborhood of one vertex that are closer to other samples in the neighborhood than they are to the vertex. As shown in [11], DPG boosts the performance of NN search considerably.

Hierarchical navigable small world (HNSW) graphs [10]. The basic idea of HNSW is inspired from the small world graph theory. The graph keeps both long range links¹ to far neighbors and short range links to close neighbors. The links are organized into an hierarchy. The long range links are maintained on the top layer while the short range links are maintained on the lower. The search is fulfilled from the top layer to the lower. According to the paper [10], such kind of coarse-to-fine search leads to “a much better logarithmic complexity scaling” than that of based on flat graph. Since the codes were released on GitHub², HNSW already receives more than 300 stars in two years. The search approach so far has been integrated with various search systems.

The hierarchical structure of HNSW is illustrated in Fig. 1. As seen from the figure, the search starts from a fixed sample on the top layer. The NN search explores the k -NN list of

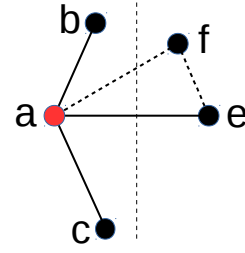


Fig. 2: The illustration of the heuristic strategy used to select the graph neighbors in the construction of HNSW graphs. An edge from a to e occludes an edge from a to f since f is closer to e than a .

a visited vertex, and descends greedily to a closer neighbor if there exists. The search on one layer stops as no closer neighbor could be found. The discovered closest neighbor on the current layer is treated as the starting point (referred as “enter point” in [10]) of the search on the lower layer. Such kind of top-down greedy search continues until it reaches to the bottom layer. On the bottom layer, the search takes the closest neighbor found from upper layer as the starting point. The search moves towards the query each time by expanding neighbors of vertices’ in a maintained top- k list. The top- k list is updated as long as any closer top- k neighbor is found. The recall of the search procedure is controlled by the parameter k (referred as ef in [10]).

The construction of HNSW graphs follows the same procedure as the NN search over it. Each candidate sample is treated as a query to query against the HNSW graphs under construction. HNSW graphs are incrementally built by repetitively inserting candidate samples into the hierarchy. The probability that a query is inserted to a layer is regularized by an exponentially decaying probability distribution. The lower the layer is, the higher of the chance that the query is inserted. On the layer that a query sample should be inserted, the discovered M neighbors are kept in the neighbors list of this sample. Accordingly, this query is possibly inserted into the neighbors lists of these M neighbors’. Parameter M controls the scale of the neighborhood list.

Graph diversification (GD) During the the insertion of a sample into the graph of each layer, the insertion operation is verified by a heuristic pruning strategy to enhance the diversity of a vertex’s neighborhood. This pruning strategy is shown in Fig. 2. As seen in the figure, the neighborhood of sample a are already inserted with samples b and c . For an incoming sample e , we check whether it should be inserted into the k -NN list of a . According to the strategy, sample e is allowed to join the neighborhood iff $m(e, a) < m(e, b)$ and $m(e, a) < m(e, c)$. Similarly given e is inserted, if another sample f is going to join the graph, its distance to a should be smaller than the distances of all joined samples (samples b , c and e) to sample f . It is imaginable that HNSW graph of each layer has been diversified by this strategy.

The motivation behind this strategy is to avoid comparing query to the cluttered neighbors, which is similar as DPG [11]. If samples are close to each other, it is redundant to visit

¹The terms “link”, “connection” and “edge” are inter-changeable across this paper.

²<https://github.com/nmslib/hnswlib>

them all since they offer similar amount of guiding information for a query. From now on, this strategy is called as graph diversification (GD). This strategy is also adopted in [18] for graph diversification.

In this paper, a hybrid scheme is presented. Namely, GD is applied on flat approximate k -NN graph. In the experiment section, we are going to show superior search performance is achieved when the NN search is conducted with the support of GD diversified flat k -NN graph.

IV. OVERVIEW OF COMPARATIVE STUDY

In our first study, graph based NN search approaches, namely HNSW [10], KGraph [8] and DPG [11] are compared to non-graph based approaches. Secondly, we investigate the role of hierarchical structure in HNSW. As claimed in [10], the hierarchy helps to skip many far neighbors in the early search. The hierarchy seemingly achieves “logarithmic complexity scaling” as B-tree does on one dimensional data. In our study, the search performance of HNSW is compared to the one based on flat graph, which is exactly the bottom layer of HNSW graphs. The hill-climbing search starts from k (referred as ef in [10]) number of random samples for both cases.

Additionally, we investigate the effectiveness of graph diversification, namely GD and DPG [11]. The performance of HNSW is compared with that of k -NN graph after GD and k -NN graph after DPG respectively. For GD graph, it is produced in three steps. Firstly, approximate k -NN graph G is built for candidate data by package KGraph provided by [8]. Graph G is further diversified by the pruning strategy introduced in HNSW (illustrated in Fig. 2). For a graph with L approximate neighbors, $L/2$ neighbors at most are selected for each sample. For diversified graph G^* , the reverse neighbors of each sample in G^* are appended to the diversified neighborhood. Since overlapping happens between the neighbors and reverse neighbors, we actually take the union of diversified neighbors and reverse neighbors. For DPG graph, we simply follow the way presented in [11]. For GD and DPG, they are built upon the same k -NN graph produced by KGraph [8].

As one will see, based on the above two experiments, the claimed “much better logarithmic complexity scaling” with hierarchical structure [10] is not observable for HNSW on high dimensional data. Another experiment is conducted to figure out the reason why HNSW and other graph based approaches fail on high dimensional case.

V. EXPERIMENTS

A. Experiment Setup

The experiments are conducted on eight datasets. Namely, there are three real-world datasets, SIFT1M, GIST1M and GloVe1M, which are popularly adopted in NN search evaluation [5], [11]. Meanwhile, five synthetic datasets are adopted. The data dimension ranges from 4 to 100. For the synthetic data, each data dimension is randomly sampled from the range [0, 1). Datasets RAND1M, GIST1M and GloVe1M are marked as challenging datasets in [11]. The search performance is measured by *Recall@1* across all our experiments. The brief

information about all eight datasets (both queries and candidate set) are summarized in Tab. I. In addition, the time costs for exhaustive NN search on each dataset are attached. l_2 -norm is adopted for all synthetic datasets, SIFT1M and GIST1M. *Cosine* distance is adopted for GloVe1M.

In general, the dimension d is higher than its real data dimension since data are embedded in a sub-space of d -dimensional space [19], [11]. The local intrinsic dimension (LID) is proposed to define the dimension of such sub-space [19]. In Tab. I, LID of each dataset estimated by [19] is shown in the 6th column. As shown in the table, LIDs of synthetic data are roughly 1.5~2.0 times lower than their data dimension d . This trend is stable. In contrast, LIDs of three real-world datasets are much lower than their data dimension and they vary considerably from one data type to another. As revealed in [11], high speed-up is achieved for graph based approaches on dataset whose ratio between d and LID is high. While the NN search is challenging when both dimension d and LID are high. To this end, we see the advantage of incorporating a series of synthetic datasets in the evaluation. By this way, the performance trend of NN search is more observable with respect to the incremental increase of real data dimension.

The approaches considered here are implemented in C++ and compiled with GCC 5.4. The experiments are conducted on workstation with CPU E5-2620 2.4GHz Xeon and 110G memory setup. The multi-threads, SIMD and pre-fetching instructions are disabled in the codes for all the approaches. There are two parameters (namely M and ef) in HNSW construction. They are optimized according to the suggestions from project *hnsplib*.

B. Graph-based versus Other Categories

In the first experiment, the performance of graph-based approaches are studied in comparison to NN search of other categories in four datasets. Three of them are marked as “hard” level in [11]. In the comparison, six representative approaches are presented, they are graph-based approaches KGraph [8], HNSW [10] and DPG [11], tree-based approaches FLANN and Annoy, LSH-based approaches SRS and quantization-based approaches PQ. Notice that DPG shares the same approximate k -NN graph as KGraph. The graph used in DPG is supplied by KGraph and further diversified. The parameter settings for all these approaches are optimized following the original papers or released package settings. Due to the big performance gap between different category of approaches, the performance is reported as the speed-up times when *Recall@1* reaches to the level of 0.8 and 0.9 respectively.

As shown in Fig 3, KGraph, HNSW and DPG show superior performance over non-graph based approaches across all the datasets. Compared to non-graph based approaches, the latent local structures of samples have been maintained in the graph. The search process is able to proceed quickly by making use of the latent structures. Among three graph based approaches, HNSW shows the best performance on SIFT1M and GIST1M, while DPG shows slightly better performance on GloVe1M and RAND1M. Another observable trend is that the

TABLE I: Overview of Datasets

Dataset	n	d	Qry	Exh. Srch (s)	LID [19]	Type
RAND10M4D	1×10^7	4	1,000	59.3	3.6	synthetic
RAND10M8D	1×10^7	8	1,000	75.6	6.5	synthetic
RAND10M16D	1×10^7	16	1,000	91.8	11.6	synthetic
RAND10M32D	1×10^7	32	1,000	147.7	19.4	synthetic
RAND1M	1×10^6	100	1,000	34.0	48.9	synthetic
SIFT1M [5]	1×10^6	128	10,000	390.1	16.3	SIFT
GIST1M [5]	1×10^6	960	1,000	283.1	38.1	GIST
GloVe1M [20]	1.2×10^6	100	1,000	37.2	39.5	Text

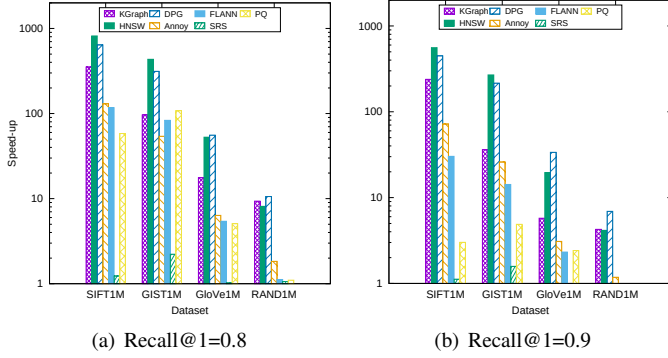


Fig. 3: Comparison between graph-based approaches and NN search of other categories.

performance drops steadily for all the considered approaches as the intrinsic dimension (LID) (shown in 6th column of Tab. I) of the data increases.

C. Hierarchy versus Flat Graph

In the second experiment, we further investigate the performance difference between NN search on HNSW graphs and NN search on flat graph. The flat graph is exactly the bottom layer of HNSW graphs. In the hierarchical search (given as HNSW), the search procedure from “hnsplib” package is adopted, which is the same as [10]. In the NN search on flat graph (given as flat-HNSW), NN-Descent procedure from KGraph package [8] is adopted. The major difference from the one used in HNSW lies in the selection of starting points. For HNSW, the starting points are supplied from non-bottom layer search. While for flat-HNSW, the starting points are random seeds.

The performance evaluation is presented in Fig. 4. The performance is reported as the recall curve against the speed-up achieved over exhaustive search. Fig. 4(a)-(e) show the performance of two approaches on synthetic random datasets. While the results on three million-level datasets from real-world are presented on Fig. 4(f)-(h).

As shown in the figure, HNSW is around two times faster than flat-HNSW on 4-dimensional data. While the efficiency of HNSW over flat-HNSW drops significantly as the data dimension reaches to 8. When the data dimension goes up as high as 32, there is no considerable difference between these two approaches. A clear trend is observable that the superiority of HNSW fades away as the dimension goes up

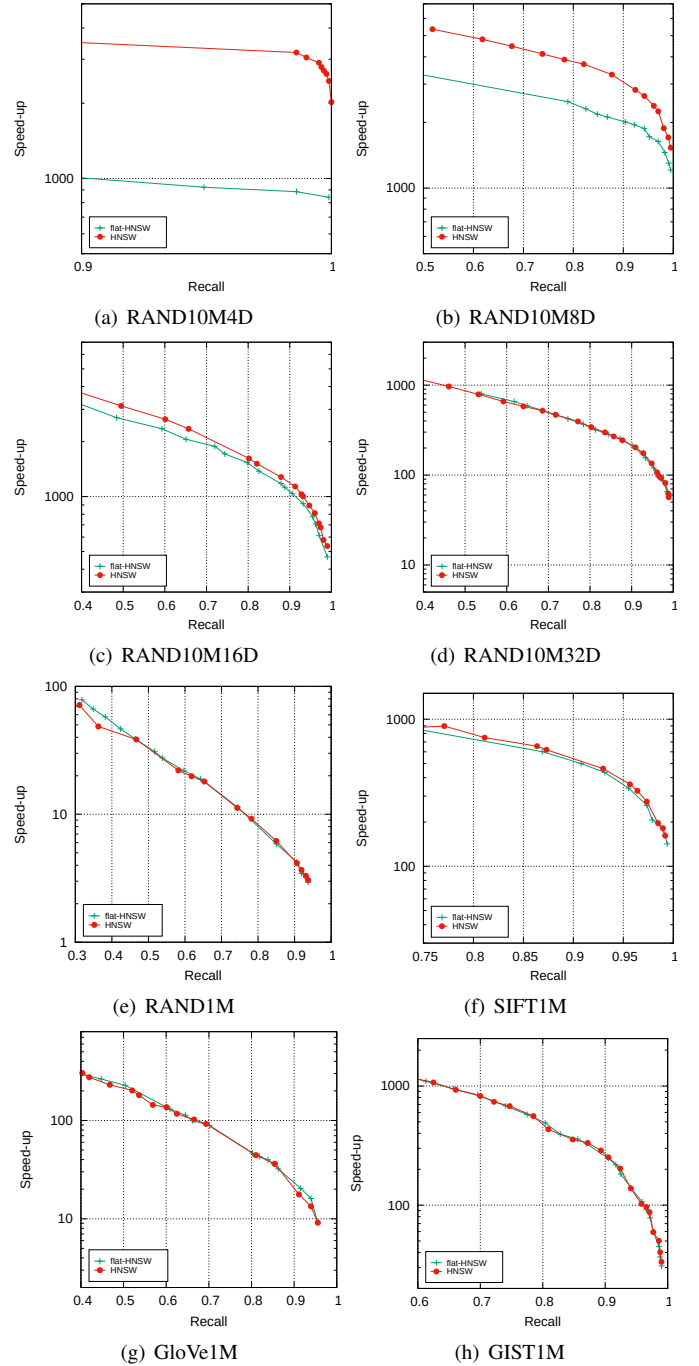


Fig. 4: The NN search performance comparison between HNSW and flat-HNSW on eight datasets.

higher. Specifically, the hierarchical search is no longer helpful when the data is beyond medium dimension (*e.g.*, 32).

On four high-dimensional datasets, the performance of HNSW and flat-HNSW is similar to each other. It is interesting to see the speed-ups on GloVe1M and RAND1M are different while they are on the same dimensionality and the same scale. The speed-up on GloVe1M is considerably higher than that of on RAND1M. This is mainly due to the difference in local intrinsic dimension. LID of GloVe1M is estimated as 39.5 as shown in Tab. I. Although this estimation is imprecise, it basically indicates the search on GloVe1M is undertaken on a much lower sub-space. As explained in [11], the approximate k -NN graph is able to capture the structure of data distribution in each neighborhood of the candidate samples. When the data distributed in the embedded sub-space, such kind of sub-space can be presented as inter-chained k -NN lists in the graph. NN-Descent therefore moves along the sub-spaces instead of exploring exhaustively the whole space, which leads to significantly higher efficiency. This is actually the advantage of graph based approaches over other categories of NN search [10], [11]. As observed from the results on real-world datasets, both HNSW and flat-HNSW are able to take advantage of the latent data distribution. The hierarchical structure in HNSW brings no extra bonus.

D. Hierarchy versus Graph Diversification

It is now clear to see hierarchical structure is not helpful for NN search on high dimensional data. In this experiment, we further study the role of graph diversification in the fast NN search. In the experiment, NN search based on HNSW graphs is treated as comparison baseline. Its performance is compared to NN search with the support of flat graph after diversification. Two diversification strategies are considered, namely GD and DPG. On the first hand, KGraph package [8] is called to produce approximate k -NN graph. In one configuration, the k -NN graph is undergone further graph diversification (GD) that is described in Section III. This configuration is given as “KGraph+GD”. In another configuration, the k -NN graph is diversified by DPG [11], which is given as “DPG” in the experiment. For reference, the NN search performance with the k -NN graph produced by [8] is also presented. It is given as “KGraph” in the experiment. So “KGraph”, “DPG” and “KGraph+GD” are built upon the same approximate k -NN graph.

As shown from Fig. 5(a)-(e), the performance trend of KGraph is in general similar as flat-HNSW on synthetic datasets. However it shows considerably worse performance than the rest approaches on the real-world datasets. While KGraph+GD demonstrates stable performance across all the datasets. In particular, it is considerably better than flat-HNSW and HNSW when dimension is greater than 8 on synthetic datasets. When the local intrinsic dimension gets higher, its performance over HNSW becomes more significant. Similar trend is also observed on real-world datasets. The major difference between KGraph+GD, DPG and KGraph lies in the incorporation of graph diversification. As seen from the figure, both GD and DPG boosts the performance of KGraph to the level of HNSW in most of the cases.

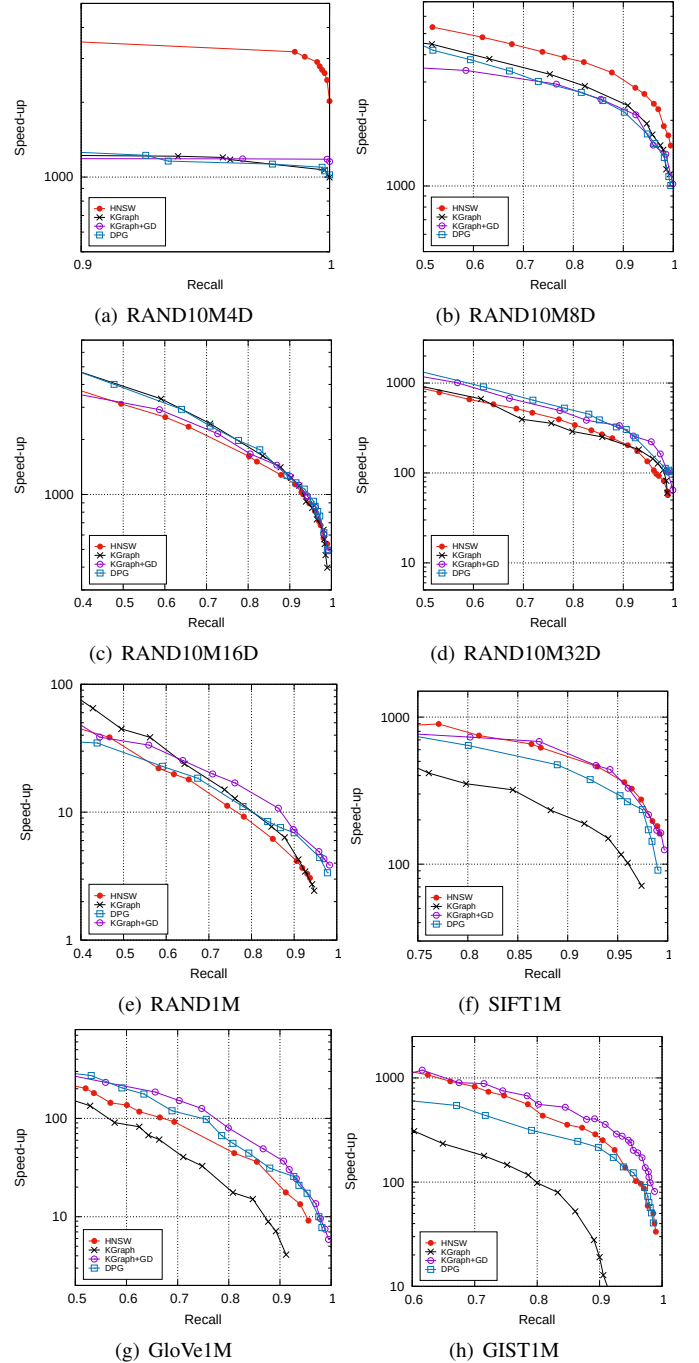


Fig. 5: The NN search performance comparison between HNSW and the one based on approximate k -NN graph on eight datasets.

To this end, we find similar performance as HNSW is achievable with flat approximate k -NN graph produced in other way around. The advantage of HNSW is only observable when the data dimension is low ($d \leq 8$). Moreover, the speed efficiency achieved in such case is only two times higher than the one based on flat graphs. Compared to HNSW, flat k -NN graph is more attractive in the sense it takes less memory to hold the structure and less time for graph construction. In terms of graph diversification operations, GD is more favorable

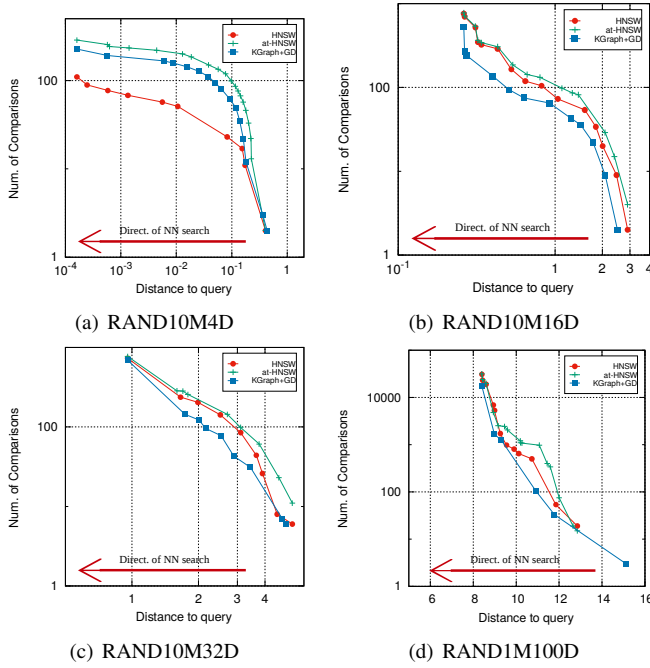


Fig. 6: The number of comparisons (y-axis) the search spends against the corresponding distance range (x-axis) the search reaches to. The observation is conducted on RAND10M4D and RAND1M. The statistics are made with 50 queries for HNSW, flat-HNSW and KGraph+GD on each dataset.

over DPG for its considerably better performance and the smaller size of the diversified graph than that of DPG.

As observed from all above experiments, the speed-up that one approach achieves drops steadily as the data intrinsic dimension increases. Graph-based approaches have no exception. On RAND1M and GloVe1M, the speed-ups from all the approaches are very limited.

VI. CURSE OF DIMENSIONALITY IN GRAPH-BASED SEARCH

Based on the above experiments, we learn that either the hierarchical structure in HNSW or graph diversification fails to return descent results on high dimensional data, such as RAND1M. In order to figure out the reason, a close investigation is carried out about HNSW and KGraph+GD as the search is conducted on four synthetic datasets with increasing dimensionality.

In the experiment, we look into the number of comparisons we need from starting samples (or seeds) to the expected closest neighbor. Specifically, we check the number of comparisons that are spent when the search process reaches to a certain distance range to the query. The statistics are shown in Fig. 6.

As shown in the figure, NN search with HNSW does move faster in the early stage of the process than those based on flat graph on both cases. This is more significant on RAND10M4D. As the search advances from distance range 0.5 to 10^{-2} , HNSW only takes less than 100 comparisons. While both flat-HNSW and KGraph+GD spend above 100

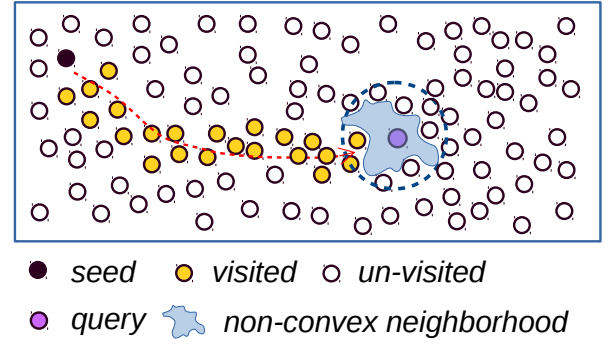


Fig. 7: The illustration of the difficulty faced by hill-climbing and the like search procedures. The search in “far neighborhood” colored by red arrow is undertaken largely efficient. While the search in “close neighborhood” is inefficient in high dimensional case. It could be as slow as an exhaustive search over the region around.

comparisons. The number of comparisons each approach spends when advancing from distance 10^{-2} to 10^{-4} is in general similar, which takes up around 15% of the total cost. As a result, the cost saved by HNSW on the early search stage becomes significant. However, the situation is largely turned around on the high dimensional case. Although it is observable that HNSW saves up less than 100 comparisons as the search advances from 18.0 to 9.6, such kind of cost-saving is marginal in contrast to the comparisons spent on “close neighborhood”. The process spends nearly 10,000 comparisons when it moves from distance 9.6 to around 8.6. Due to the high cost in “close neighborhood” search for all the approaches, there is no big performance difference among three graph based approaches.

This phenomenon is interpreted in Fig. 7. Please be noted that the “close neighborhood” is usually a non-convex region. As the search reaches to the “close neighborhood”, it could be trapped in a local optima when the maintained top- k list is short. Otherwise, when the top- k list is sufficiently long, it might search around the “close neighborhood” exhaustively. As a result, it leads to either high efficiency while with low recall or high recall while with low efficiency. In the low dimensional case, this will not be a big problem as samples in this neighborhood reside in the inter-chained NN lists. The hill-climbing therefore converges quickly. In this case, the number of comparisons in “far neighborhood” takes a large portion. As a result, the time cost saved by the hierarchical search becomes significant. In the high dimensional case, samples in the “close neighborhood” may reside in the NN lists that are isolated from each other. They may be relatively close to each other however hardly overlap with their neighbors. It is therefore hard to guide the search to jump from one local optima to another. As the data dimension goes up higher, this problem becomes more apparent and serious. As a consequence, when both the data dimension and intrinsic data dimension are high, HNSW, as other graph based approaches, becomes inefficient.

VII. CONCLUSION

We have presented our comparative studies over two major graph-based approaches. Three major issues are investigated. Firstly, we find that the hierarchical structure of HNSW only shows superior performance over flat graph on low dimensional data. The speed-efficiency over the approaches without hierarchy support is around two times to its best. The advantage that the hierarchical structure brings fades away as the dimension goes up as high as around 32. Moreover, it is possible to achieve similar or even higher speed efficiency over HNSW with the support of approximate k -NN graph, after it has been undergone graph diversification. Furthermore, we find that most of the graph-based approaches face the same difficulty when both data dimension and the intrinsic data dimension are high. This difficulty is directly linked to “curse of dimensionality”. Neither the hierarchical structure nor the graph diversification scheme, which are effective in many cases, are able to address this issue.

ACKNOWLEDGMENT

This work is supported by National Natural Science Foundation of China under grants 61572408.

REFERENCES

- [1] J. L. Bentley, “Multidimensional binary search trees used for associative searching,” *Communications of ACM*, vol. 18, pp. 509–517, Sep. 1975.
- [2] A. Guttman, “R-trees: A dynamic index structure for spatial searching,” in *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*, vol. 14, (New York, NY, USA), pp. 47–57, ACM, Jun. 1984.
- [3] M. Muja and D. G. Lowe, “Scalable nearest neighbor algorithms for high dimensional data,” *Trans. PAMI*, vol. 36, pp. 2227–2240, 2014.
- [4] C. Silpa-Anan and R. Hartley, “Optimised kd-trees for fast image descriptor matching,” in *CVPR*, 2008.
- [5] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *Trans. PAMI*, vol. 33, pp. 117–128, Jan. 2011.
- [6] A. Babenko and V. Lempitsky, “Efficient indexing of billion-scale datasets of deep descriptors,” in *CVPR*, pp. 2055–2063, 2016.
- [7] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, “Locality-sensitive hashing scheme based on p-stable distributions,” in *Proceedings of the Twentieth Annual Symposium on Computational Geometry*, (New York, NY, USA), pp. 253–262, ACM, 2004.
- [8] W. Dong, C. Moses, and K. Li, “Efficient k-nearest neighbor graph construction for generic similarity measures,” in *Proceedings of the 20th International Conference on World Wide Web, WWW’11*, (New York, NY, USA), pp. 577–586, ACM, 2011.
- [9] K. Hajebi, Y. Abbasi-Yadkor, H. Shahbazi, and H. Zhang, “Fast approximate nearest-neighbor search with k-nearest neighbor graph,” in *International Joint Conference on Artificial Intelligence*, pp. 1312–1317, 2011.
- [10] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *Trans. PAMI*, pp. 1–1, 2018.
- [11] W. Li, Y. Zhang, Y. Sun, W. Wang, W. Zhang, and X. Lin, “Approximate nearest neighbor search on high dimensional data-experiments, analysis and improvement,” *IEEE Transactions on Knowledge and Data Engineering*, pp. 1–1, 2019.
- [12] Y. A. Malkov, A. Ponomarenko, A. Lovinov, and V. Krylov, “Approximate nearest neighbor algorithm based on navigable small world graphs,” *Information Systems*, 2013.
- [13] B. Naidan, L. Boytsov, and E. Nyberg, “Permutation search methods are efficient, yet faster search is possible,” *Proceedings of the Vldb Endowment*, vol. 8, no. 12, pp. 1618–1629, 2015.
- [14] E. Bernhardsson, “Annoy: approximate nearest neighbors in c++/python optimized for memory usage and loading/saving to disk,” 2016.
- [15] M. Datar, P. Indyk, N. Immorlica, and V. S. Mirrokni, “Locality-sensitive hashing scheme based on p-stable distributions,” in *Twentieth Symposium on Computational Geometry*, 2004.
- [16] W. Liu, J. Wang, R. Ji, Y. G. Jiang, and S. F. Chang, “Supervised hashing with kernels,” in *CVPR*, 2012.
- [17] Y. Sun, W. Wang, J. Qin, Y. Zhang, and X. Lin, “SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny,” in *The Proceedings of the VLDB Endowment*, pp. 1–12, Sep. 2014.
- [18] B. Harwood and T. Drummond, “FANNG: Fast approximate nearest neighbour graphs,” in *CVPR*, pp. 5713–5722, 2016.
- [19] E. Levina and P. J. Bickel, “Maximum likelihood estimation of intrinsic dimension,” *Advances in Neural Information Processing Systems*, 2005.
- [20] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” in *Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543, 2014.